ECE470 Lab 4 Report

Sidharth Nagarajan

November 2024

1 Introduction and Objective

The goal of this project is to use the UR3 Robot to draw a picture based on an input image. The robot should be able to replicate the image with reasonable accuracy in less than 15 minutes.

2 Methodology

2.0.1 Image Processing

To simplify the problem, we are ignoring coloring and shading the image. Our goal is only to draw the important outlines of the image such that it is recognizable. To draw these outlines, we need keypoints. We won't ever be drawing a truly continuous line in this project. All the lines drawn are straight lines between keypoints. Even in the image itself, which is represented by pixels, the lines that appear are also simply straight lines between keypoints (pixels). This section will focus on how the keypoints were acquired.

All of the image processing done in this project heavily relies on the OpenCV Python Library. OpenCV reads digital images as 3d numpy arrays, of dimensions M x N x 3. Where M and N are the image height and width in pixels respectively, and 3 corresponds to the RGB value of each pixel. OpenCV has a convenient function called Canny, which performs Canny Edge Detection on an image. Edge detection finds locations in an image where there is a difference between different colored pixels. This provides us with an outline of the image as shown in Figure 1.

Now we have an image with only black and white pixels, where the black pixels are the lines we have to draw. However, not all the pixels are connected, so we can't simply choose a starting pixel and move the marker to all the other pixels. We somehow have to determine which sets of pixels form lines. OpenCV offers another convenient function for contours. These functions join all the continuous points along a color boundary in the image. For example, in our picture, there are multiple color boundaries between black and white pixels. Contours finds sets of these boundary pixels that form a continuous line, and appends them to a python tuple. An example of the contour function is shown

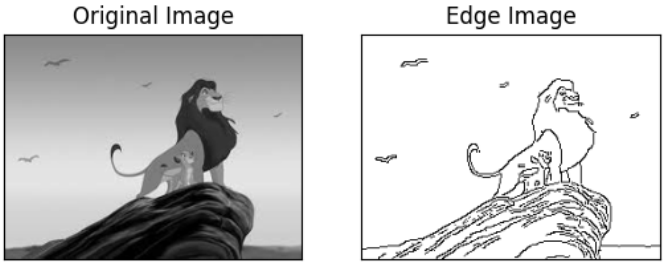

Figure 1: Edge Detection Example

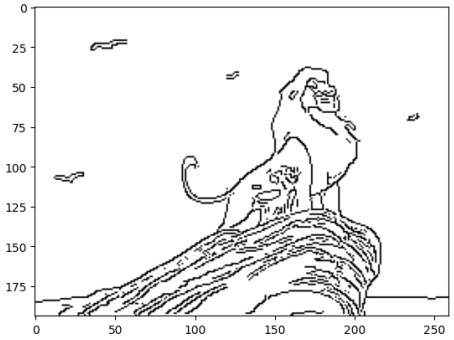

Figure 2: Contour Function

in Figure 2. Now we have a list of all the individual lines that need be drawn, and each of the keypoints for said line.

The next problem is reducing the total number of points to draw. Higher resolution images are very pixel dense, and we lose a lot of time drawing each and every contour in the contour tuple, and every point for each contour. On the actual paper, the human eye won't be able to distinguish between two pixels right next to each other. Therefore, we only draw the largest contours (longest lines), and only every 12th pixel.

2.1 Image Scaling and Transformation

Now that we have the set of keypoints to draw and the order to draw them in, we can map the image to a physical space. We are using standard 8.5 x 11 inch printer paper to draw on, which is always placed in the same location for each drawing. We can do some quick scaling to make any image a square with 250 x 250 pixels using an OpenCV function. Then we choose a square area on our paper to encompass the image area. In our case, we chose 6 x 6 inches. We use a coordinate transformation based on how many camera pixels correspond to a certain distance in the real world.

2.2 Drawing the image

Now all that's left to do is command the UR3. We loop through all the relevant contours and points, lifting the marker up between each contour so as to not draw extra lines. The proper height in the z direction was determined based on how high/low the maker was placed on the end-effector attachment. The robot position was determined based on joint angles acquired from inverse kinematics based on the transformed XY position of the pixel and the marker height Z.

3 Results

Our code was able to successfully draw both outlined and fully colored images with no manual steps between inputting the image and drawing.